

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

ILIA FILONENCO

Group FAF 253

Laboratory Report

Object Oriented Programming

Laboratory Work 1

Logging Library and Parentheses Validator

Programming language Python

Chisinau 2026

1 Program Structure

The program implements a logging library and uses it to validate round parentheses. Logger filters and formats messages. LogLevel defines four ordered levels: TRACE, INFO, WARN and ERROR. ParenthesesValidator checks the input and reports events through a Logger named Parentheses. The validator contains a logger; it does not inherit from Logger. main.py configures the application and runs the examples.

2 Classes and Objects

Logger is a class describing the state and behaviour of a logging object. Logger("Parentheses") creates an instance whose _name stores its individual name. The name property exposes that value for reading. The leading underscore marks an internal attribute by Python convention; it does not enforce private access.

A ParenthesesValidator object stores its logger in _logger. The balance counter is local to is_valid(), so each validation starts from zero. This allows the same validator object to process several strings independently.

3 Static and Instance Members

Logger.level and Logger.format are class attributes shared by all loggers. Each instance has its own _name. The static methods set_level() and set_format() update the class attributes without needing self. The following example changes the settings for both existing objects:

```
first = Logger("First")
second = Logger("Second")
Logger.set_level(LogLevel.ERROR)
Logger.set_format("%n: %m")
first.info("Hidden")       # no output
second.error("Failure")   # Second: Failure
```

log() reads Logger.level directly. TRACE, INFO, WARN and ERROR have values 10, 20, 30 and 40. A message is printed when its value is at least the configured threshold. At INFO, TRACE is hidden, while INFO, WARN and ERROR remain visible.

4 Methods and Responsibilities

log() checks the threshold, prepares the time, name, level and message, replaces the format placeholders, and prints the result. The trace(), info(), warn() and error() methods call log() with a fixed level. For example, self._logger.info(...) in is_valid() reports a matching pair. Keeping output in Logger lets the validation algorithm remain independent of formatting rules.

The format supports %t, %m, %n and %l. The bonus %r selects a random phrase associated with the level of the current entry. re.sub() replaces placeholders in one pass, so placeholder-like text inside a message is preserved.

5 Testing and Correctness

An opening parenthesis increases balance. A closing parenthesis requires a positive balance and then decreases it. Otherwise, the method returns False immediately. At the end, balance must be zero. The counter tracks unmatched opening parentheses in every processed prefix. The algorithm takes O(n) time and O(1) extra space, excluding log output.

The four required cases produced the expected results: () returned True;)((())) returned False; (returned False; and (())((()())()) returned True. Empty input is valid. Characters other than round parentheses are rejected.

All eight automated tests passed. They cover level filtering, shared settings, formatting, %r, required log events and invalid API arguments. All 511 parenthesis strings of lengths 0 to 8 also matched an independent stack-based reference. The case)(prevents accepting equal counts in the wrong order. Validation results remain correct at every logging level.

6 Reflection

The main OOP concept demonstrated here is separation of responsibilities through composition. ParenthesesValidator performs validation and delegates messages to Logger. Shared configuration and individual names show the difference between class and instance state. A possible improvement is to accept a logger as a constructor argument, allowing another output destination or a test substitute without changing the validation algorithm.